

Interim Progress Report

Numerical Evaluation of Multi-Agent Roles (OpenHands Setup)

To: Professor Sergey Kovalchuk

From: Ali Shahab

Date: June 10, 2026

Dear Professor Kovalchuk,

Following up on my previous update, I have successfully configured the OpenHands framework inside a Docker environment and begun the numerical evaluation phase using the HumanEval benchmark.

The goal of today's testing was to evaluate how different "Role Configurations" (Baseline Coder vs. Pair Programming) impact success rates, and whether the underlying LLM's parameter size affects its ability to manage these roles.

Methodology

- **Framework:** OpenHands (Local Docker Deployment)
- **API Provider:** OpenRouter
- **Benchmarks Used:** HumanEval (#0 and #1)
- **Configurations Tested:**
 - **Setup A:** Baseline (Coder Agent Only)
 - **Setup B:** Pair Programming (Coder + Tester Agents acting sequentially in one turn)

Preliminary Data (Today's Results)

I recorded the pass/fail rates and noted the specific types of failures that occurred:

Key Findings from Today

1. **Framework-Level Failure vs. Logic Failure:** The mid-tier models (like Qwen-7B and poolside/laguna) did not fail due to poor Python logic. They failed because of **Syntax Degradation**. When tasked with strict tool-calling schemas (XML), smaller models hallucinated the tags (e.g., `<parameter=command>`), causing the framework to reject the files entirely.

Task ID	Model Size	Role Configuration	Result	Failure Type	Debugging
HumanEval #0	Mid-Tier (7B)	Setup A (Coder Only)	FAIL	Framework (Broken XML)	1 (Failed)
HumanEval #0	Massive (120B)	Setup A (Coder Only)	PASS	None	0
HumanEval #1	Mid-Tier (7B)	Setup B (Coder + Tester)	FAIL	Framework (Broken XML)	1 (Failed)
HumanEval #1	Massive (120B)	Setup B (Coder + Tester)	PASS	None	0

2. **The Impact of Role Density:** When the cognitive load was increased in Setup B (asking the agent to write code *and* write tests *and* execute bash commands), the smaller models experienced severe “attention collapse,” failing to communicate with the OpenHands framework completely.
3. **Parameter Scaling Resolves Bottlenecks:** Switching to a massive parameter model (gpt-oss-120b) completely eliminated the framework failure. The 120B model successfully juggled the high Role Density of Pair Programming (writing code, generating tests, and verifying them in the bash terminal) with perfect XML syntax on the first attempt, yielding a 100% pass rate.

Appendix: Experimental Prompts Used Today

To ensure strict methodological control, the following standardized prompts were injected into the OpenHands environment for the trials:

Setup A (Coder Only - HumanEval #0)

“You are a Coder agent. Your ONLY job is to write code. Do not create a task list, do not write test scripts, and do not execute terminal commands. Please use your code editor tool to write a Python file named `humaneval_0.py` in the `/workspace` directory that solves this exact problem:

Task: Check if in a given list of numbers, any two numbers are closer to each other than a given threshold.

Signature: `def has_close_elements(numbers: List[float], threshold: float) → bool:`

Example: `has_close_elements([1.0, 2.0, 3.0], 0.5) → returns False`”

Setup B (Pair Programming - HumanEval #1)

“You are a Pair Programming team (Coder + Tester). Do not create a task list. Please use your code editor tool to write a Python file named `humaneval_1.py` that solves the following problem. After saving it, act as the Tester: use your code editor tool to write a separate script called `test_humaneval_1.py` to test the logic, and then execute it in the bash terminal to verify your code passes.

Task: Separate a string of parentheses into a list of balanced parenthesis groups.

Signature: `def separate_paren_groups(paren_string: str) → List[str]:`

Rules: Ignore any spaces in the input string. A balanced group of parentheses is one where every open parenthesis ‘(’ has a matching close parenthesis ‘)’.”

I will continue expanding this dataset with more complex tasks in the coming weeks.

Best regards,

Ali Shahab